

FREQUENTLY ASKED **50+ ADVANCED CDC** **INTERVIEW QUESTIONS**

Your MUST have Interview
Guide to crack
TOP PRODUCT VLSI COMPANIES



MUST HAVE CDC INTERVIEW HANDBOOK

Q1: What is Clock Domain Crossing (CDC) and why is it critical in digital design?

Answer:

CDC refers to the transfer of data between two flip-flops that are driven by unrelated or asynchronous clocks. It's critical because:

- Asynchronous domains can lead to metastability.
- Improper CDC handling may cause data loss, glitches, or timing violations.
- Increasing SoC complexity introduces multiple clock domains (performance, power gating, IP reuse), increasing CDC issues.

Q2: Explain metastability in the context of CDC. How do you mitigate it?

Answer:

Metastability occurs when a flip-flop's input changes close to its clock edge, violating setup/hold timing. The output becomes unpredictable or unstable.

Mitigation Techniques:

- Double-flop synchronizer (for control signals).
- Handshake mechanisms or asynchronous FIFOs (for data buses).
- Use of Gray coding (for counters).
- CDC verification tools to detect paths and check proper synchronization.

Q3: Describe the architecture and use-case of a 2-flop synchronizer.

Answer:

A 2-flop synchronizer is used to safely transfer a single-bit control signal from one clock domain to another.

clk_B domain:

```
flop1 <= async_signal;      // first flop samples  
signal  
flop2 <= flop1;             // second flop resolves  
metastability
```

Use-case: Safe control/status signal transfer between domains with minimal latency and area.

Q4: Why can't we use 2-flop synchronizers for multi-bit buses?

Answer:

Because each bit of a bus may become metastable independently, leading to inconsistent values across bits. 2-flop synchronizers don't ensure data coherency.

Solution: Use:

- Asynchronous FIFO
- Handshake protocols
- Gray-coded counters (in case of address transfer)

Q5: What is an asynchronous FIFO? How does it manage CDC?

Answer:

An asynchronous FIFO allows data transfer between write and read clock domains using:

- Dual-port RAM (write on one clock, read on another).
- Gray-coded write and read pointers.
- Pointer synchronization logic using 2-flop synchronizers.
- Full/empty logic comparing pointers.

This ensures safe and lossless multi-bit data transfer across CDC.

Q6: What are common CDC issues caught in STA and those missed in STA?

Answer:

Detected by STA:

- CDC path considered synchronous by mistake.
- Violations due to incorrect false path constraints.

Missed by STA:

- Metastability.
- Pulse stretching/missing pulses.
- Incorrect handshake protocol.
- Data incoherency in buses.

Q7: How is Gray coding useful in CDC? Where is it used?

Answer:

Gray coding ensures only one bit changes at a time between consecutive values, reducing risk of inconsistent multi-bit reads.

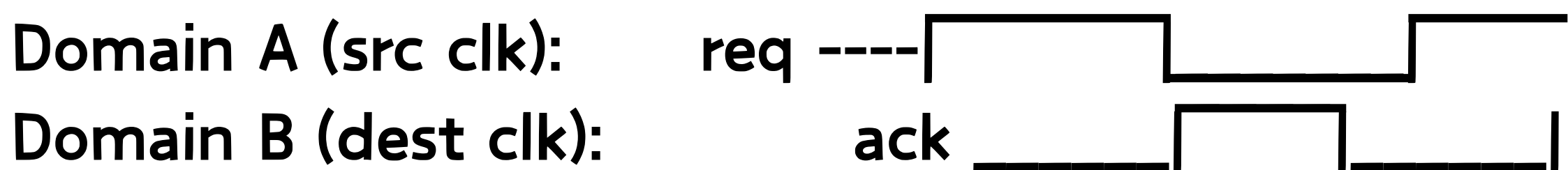
Used in:

- FIFO pointers (read/write address).
- Token passing and status signaling.

Q8: Explain handshake-based CDC mechanism with a timing diagram.

Answer:

A request-acknowledge handshake ensures safe single data transfer:



Steps:

- 1.req asserted in source domain.
- 2.req synchronized and detected in destination.
- 3.Dest domain latches data, raises ack.
- 4.Source sees ack, lowers req.
- 5.Dest lowers ack.

Q9: What are CDC verification techniques used in industry?

Answer:

- **Static CDC Analysis Tools:**
 - **Synopsys SpyGlass CDC**
 - **Mentor Questa CDC**
 - **Cadence JasperGold CDC**
- **Formal Verification for protocol correctness.**
- **Code review and design guidelines.**
- **Assertions (SVA) for handshake protocols.**

Q10: What is pulse synchronizer and when is it preferred?

Answer:

Used to transfer a narrow pulse across clock domains safely.

Mechanism:

- **Pulse is stretched to at least 2 destination clock cycles.**
- **Synchronized via double flop.**
- **Dest domain detects rising edge of pulse.**

Use-case: One-shot pulse transfer (reset, enable, start).

Q11: Why are CDC issues hard to reproduce in simulation?

Answer:

- Simulation has fixed delta delays and ideal flip-flop models.
- Metastability is analog in nature, not captured in digital sims.
- CDC bugs may appear only in corner cases or due to race conditions.

Thus, formal/static CDC analysis is a must.

Q12: How do you handle reset signals crossing clock domains?

Answer:

- Use synchronous reset locally.
- Cross reset using 2-flop synchronizer.
- Avoid asynchronous reset propagation across domains.

Or use a reset synchronizer circuit with glitch-free assertion/deassertion.

Q13: What are CDC-aware RTL coding practices?

Answer:

- Avoid direct sampling of async signals.
- Use synchronizers, handshake, or FIFOs.
- Isolate combinational logic between clock domains.
- Add comments/hints to assist CDC tools.
- Clearly name signals as `*_async`, `*_sync`, `*_cdc`.

Q14: How do CDC tools detect metastability issues?

Answer:

- Identify data/control signals crossing from one clock domain to another.
- Check if proper synchronization logic (2-flop, FIFO) is used.
- Flag violation if:
 - Flop is missing.
 - 2-flop not in same clock domain.
 - Unprotected bus transfer.

Q15: What is data coherency? How do you ensure it in CDC?

Answer:

Data coherency ensures all bits of a bus are transferred together without skew.

Ensured by:

- Handshake: Confirm one transfer at a time.
- FIFO: Stores entire data word.
- Gray-coded pointers: Safe address tracking.
- Data-valid signal: Indicates when data is ready.

Q16: What is a CDC false path? How do you handle it in synthesis/STA?

Answer:

A CDC false path is a timing path between two different asynchronous clock domains which should not be analyzed for setup/hold in STA.

Handling:

- Declare as `set_false_path` in SDC constraints.
- Inform tools to ignore setup/hold checks for

Q16: What is a CDC false path? How do you handle it in synthesis/STA?

Answer:

A CDC false path is a timing path between two different asynchronous clock domains which should not be analyzed for setup/hold in STA.

Handling:

- Declare as `set_false_path` in SDC constraints.
- Inform tools to ignore setup/hold checks for that path.

Q17: What is reconvergence pessimism in CDC?

Answer:

Reconvergence pessimism occurs when multiple synchronized versions of a signal are used, and then they reconverge into the same logic block.

This leads to:

- CDC tools treating them as independent, not correlated.
- Potential for glitch or hazard due to different sync delays.

Solution:

- Use only one synchronizer output in downstream logic.
- Avoid reconvergence by using same sync output across modules.

Q18: Describe a CDC scenario that leads to data loss.

Answer:

Example scenario:

- Source domain sends a pulse that lasts 1 clk cycle.
- Destination domain's clock is slower, and it misses the pulse.

Result: Pulse never sampled → data loss.

Solution:

- Use a pulse stretcher or toggle synchronizer.
- Convert 1-cycle pulse to level signal valid for multiple dest clock cycles.

Q19: How can you design a glitch-free clock gating signal across domains?

Answer:

Glitch-free clock gating requires that:

- The enable signal is synchronized to the gated clock domain.
- Use 2-flop sync before applying to clock gating cell.
- Alternatively, use scan-enable friendly gated clock cell from library.

Never directly gate a clock with async control signal –leads to glitch and simulation mismatch.

Q20: Explain the role of FIFO full and empty signals in CDC.

Answer:

In asynchronous FIFOs:

- Full signal: Indicates write pointer has caught up to read pointer (no more space).
- Empty signal: Read pointer has caught up to write pointer (no data).

These signals:

- Are derived using gray-coded pointers.
- Are synchronized across domains using 2-flop sync.

Incorrect sync = wrong full/empty = data corruption.

Q21: What is a toggle synchronizer? How does it differ from a pulse synchronizer?

Answer:

Toggle Synchronizer:

- Source toggles a bit ($0 \rightarrow 1 \rightarrow 0 \dots$) every event.
- Destination synchronizes and detects toggle change.
- Safer than narrow pulses.

Differences:

- Toggle is level-based transition, pulse is duration-based.
- Toggle ensures dest can catch transition reliably.

Q22: How do you verify CDC logic in RTL?

Answer:

- Use CDC static analysis tools (SpyGlass CDC, Questa CDC).
- Add assertions in SystemVerilog (e.g., for handshake success).
- Manually check:
 - Synchronizer presence.
 - Clock domain boundaries.
 - No comb. logic between domains.
- Use lint rules for naming conventions: `_async`, `_sync`.

Q23: What are some RTL design rules to ensure CDC correctness?

Answer:

- Avoid comb logic between flop boundaries across domains.
- Use sync modules (pre-designed and reviewed).
- Name all signals with clear intent: `*_async`, `*_sync`.
- Isolate buses with control signals and handshake.
- Document and constrain CDC paths.

Q24: How do you handle clock domain crossing for scan enable (scan_en) signals in DFT?

Answer:

- scan_en must be synchronized into each clock domain.
- Typically uses 2-flop sync.
- Avoids scan-related CDC issues during test mode.
- Alternatively, use synchronous scan enable generators per domain.

Q25: What is "data loss due to metastable resolution delay"?

Answer:

Metastable resolution may take longer than 1 clock cycle. If the signal is sampled before it stabilizes:

- Destination gets wrong data.
- Could cause protocol violation or corrupted transaction.

Solution: Use 2 or more flops in sync chain → give metastability time to settle.

Q26: How is bus data integrity maintained across CDC?

Answer:

- Use FIFO or handshake.
- Latch data into a register buffer before crossing.
- Use data_valid signals.
- Transmit one word per handshake → prevent bit skew.

Never use multiple 2-flop syncs for each bit in a bus.

Q27: Describe a real-case CDC bug you handled.

Answer:

Example:

In a design with async reset release from system clock, reset wasn't synchronized to the local domain. This caused random flip-flop initialization issues after power-up.

Fix:

- Added reset synchronizer per domain.
- Synchronized deassertion of reset.
- Added RTL assertions to catch reset glitches.

Q28: What is the role of reset synchronizers in CDC, and how are they designed?

Answer:

Reset synchronizers ensure that asynchronous reset deassertion is safely aligned with the destination clock domain.

Design:

- Use two flip-flops clocked by the destination clock.
- Connect async reset as the reset pin of the flops.
- Ensure glitch-free, synchronous deassertion.

It's a critical design element to avoid random logic initialization across multiple domains.

Q29: Why is it important to synchronize both request and acknowledge in handshake CDC?

Answer:

In handshake CDC:

- If req is not properly synchronized into the destination domain, it might be missed.
- If ack is not synchronized back, the source may assert a second request before the first is complete.

Both must be synchronized to their respective domains to maintain protocol correctness and avoid data corruption.

Q30: How can you identify and avoid combinational logic between two asynchronous domains?

Answer:

Identification:

- Trace logic paths between source and destination clocks.
- Use CDC tools to flag combinational paths across domains.

Avoidance:

- Insert synchronizers or handshake modules.
- Ensure that the clock domain boundary is always through a registered interface, not combinational logic.

This prevents glitches and race conditions.

Q31: What are the typical components of a CDC verification report?

Answer:

CDC verification tools generate reports containing:

- List of CDC paths and source/destination clocks.
- Detected violations (missing synchronizers, reconvergence, glitches).
- Suggestions for fixing issues.
- Block-wise or hierarchical categorization.
- Path classification (control, data, handshake).

These help designers debug and refine CDC logic.

Q32: What are some CDC-specific bugs that are hard to simulate but show up in silicon?

Answer:

- Metastability-induced logic failure not seen in functional simulation.
- Pulse loss due to clock skew or narrow control pulses.
- Incorrect FIFO full/empty status from improper pointer sync.
- Data inconsistency in bus transfers.
- False reset release in one domain while others stay in reset.

These are best caught by static CDC tools, not simulation.

Q33: How do you design a CDC interface between a fast and a slow clock domain?

Answer:

- Use asynchronous FIFO to handle differing data rates.
- If sending a pulse or flag: use pulse/toggle synchronizer.
- Ensure data is latched and held long enough for the slower clock to detect.

Design should account for possible overflow or data hold scenarios.

Q34: Why are CDC paths considered false by default in STA?

Answer:

Since clocks in different domains are asynchronous, their timing relationship is undefined.

- Static Timing Analysis assumes a fixed relationship, which doesn't exist in CDC.
- To avoid misleading violations, these paths are marked as false paths using constraints.

Timing is handled via robust sync logic, not STA.

Q35: How do you avoid CDC issues when reusing IPs with different clock domains?

Answer:

- Use well-defined interfaces (FIFO, handshakes).
- Define clock domain boundaries clearly.
- Ensure synchronization logic is part of the IP wrapper or integration layer.
- Validate with CDC tools post-integration.

Q36: How would you transfer a burst of data between two clock domains?

Answer:

Best methods:

- Asynchronous FIFO: most efficient for bursts.
- Use valid-ready handshake if FIFO is not suitable.
- Ensure burst size is manageable within FIFO depth.

Q37: What is the danger of directly sampling a signal from another clock domain?

Answer:

Direct sampling without synchronization can lead to:

- Metastability
- Incorrect data
- Glitches in combinational logic
- Unpredictable simulation behavior

Always pass signals through sync chains, handshake mechanisms, or FIFOs.

Q38: What role does 'data_valid' signal play in a CDC transfer?

Answer:

- Indicates to the receiver that the multi-bit data is stable and ready.
- Helps in avoiding partial or incoherent data sampling.
- Must be synchronized using 2-flop before use in destination domain.

It is essential for ensuring safe data reception across domains.

Q39: Can scan chains introduce CDC issues during test mode?

Answer:

Yes, because scan chains typically span across multiple clock domains:

- scan_en and scan_clk need to be synchronized per domain.
- If not handled, they can create glitches or corruption in the scanned data.

Proper test insertion tools manage scan CDC with synchronization cells.

Q40: What's the importance of CDC naming conventions in RTL coding?

Answer:

Clear naming helps:

- Tools and designers identify CDC signals easily.
- Enforce lint rules and CDC checks.
- For example: suffixes like _async, _sync, _cdc hint at the signal's nature.

Improves maintainability and readability of the design.

Q41: How can CDC issues affect power-aware designs with clock gating?

Answer:

- Improper sync of clock gating control can cause glitches, leading to extra power consumption.
- If clock-gated domain wakes asynchronously, sync issues can arise.
- Must synchronize enable signals and control logic properly across domains.

CDC correctness is key in low-power architectures.

Q42: What do you check in a design review for CDC correctness?

Answer:

Checklist:

- All async crossings use synchronizers or handshakes.
- No combinational logic across domains.
- Proper use of async FIFOs for buses.
- Resets are properly synchronized.
- CDC report from tools is clean or issues are justified.

Manual review complements tool-based analysis.

Q43: What's a handshake-based CDC? Why is it reliable for data transfer?

Answer:

A handshake-based CDC involves a request-acknowledge protocol between the source and destination domains.

Why it's reliable:

- Each data transfer is confirmed.
- It tolerates clock frequency mismatches.
- Synchronization is done on both control signals (req, ack).
- Prevents data loss or duplication.

Often used for low-speed or control signal transfer.

Q44: What happens if you miss synchronizing an acknowledge signal in handshake CDC?

Answer:

- The source may believe the request was not acknowledged and resend data.
- Results in duplicate transfers, incorrect state machines, or data corruption.
- Always synchronize both req and ack to avoid such CDC bugs.

Q45: What are the common mistakes when you are implementing 2-flop synchronizers?

Answer:

- Using combinational logic between the flops.
- Resetting the synchronizer flops unnecessarily (only required at startup).
- Clocking the sync flops with the wrong domain's clock.
- Instantiating sync logic manually instead of using standard modules.

Such mistakes reduce the effectiveness of metastability resolution.

Q46: Can metastability be completely eliminated in CDC?

Answer:

No, metastability cannot be completely eliminated in asynchronous crossings.

But:

- It can be mitigated using 2 or more flops.
- Reduced by designing proper synchronization mechanisms.
- Ensuring slower toggle rates also helps improve reliability.

Designs focus on reducing the probability, not eliminating it.

Q47: When is handshake preferred over FIFO in CDC?

Answer:

Use handshake when:

- Transferring occasional data or flags.
- Area and power are more critical than throughput.
- You need tight control over the exact transaction flow.

For high-throughput or continuous data, FIFO is preferred.

Q48: What happens if asynchronous FIFO pointers are not gray-coded?

Answer:

- Binary pointers can glitch when multiple bits change simultaneously.
- This causes incorrect full/empty detection in the destination domain.
- Can lead to data overwrite or read underflow.

Gray coding ensures only one bit changes at a time
→ safe pointer sync.

Q49: What type of CDC errors might you see during post-silicon bring-up?

Answer:

- Sporadic data loss or corruption.
- Unstable or inconsistent status flags.
- Glitchy control behavior due to missed sync.
- Unexpected state machine behavior during clock switching.

These issues often don't appear in simulation → require static CDC analysis before tape-out.

Q50: What is a synchronizer hard cell and why use it instead of two regular flops?

Answer:

A synchronizer hard cell is a library-provided hardened synchronizer circuit:

- Optimized for metastability recovery.
- Physically placed to minimize routing delay.
- Often has MTBF guarantees.

Preferred for critical control signals where reliability is paramount.

Q51: Is it safe to use a single-flop synchronizer in any scenario?

Answer:

No, a single-flop synchronizer does not provide sufficient time for metastability resolution.

Only acceptable in special cases like:

- Very slow toggle rate + high MTBF tolerance.
- Not recommended in production silicon.

Best practice: always use at least 2-stage sync.

Q52: What are CDC risks when using gated clocks or derived clocks?

Answer:

- Derived clocks can introduce unclear clock domain boundaries.
- Gated clocks may lead to data glitches or missed edges in sync logic.
- Tools may fail to detect these as CDC if the clocks appear related.

Always treat gated/derived clocks with caution and verify them using CDC tools.

Q53: Why is full RTL simulation insufficient for CDC verification?

Answer:

Because:

- RTL simulation does not model metastability.
- Clocks are often derived from the same base in simulation → doesn't reflect async behavior.
- Glitch and race conditions won't manifest unless specifically modeled.

Hence, use dedicated CDC static tools for accurate analysis.

Q54: Can scan insertion tools break CDC paths?

Answer:

Yes, improperly inserted scan logic may:

- Split sync chains.
- Introduce muxes or logic that alter timing.
- Create paths that don't exist in functional mode.

Use DFT-aware CDC verification after scan insertion to catch these.

Q55: What precautions should be taken when crossing reset signals across domains?

Answer:

- Deassertion must be synchronized to the destination domain.
- Assertion can remain asynchronous.
- Avoid using reset as a data signal.
- Always synchronize using 2-flop reset synchronizer to avoid partial release.

Q56: What are "bad CDC practices" you commonly review during code review?

Answer:

- Direct sampling of async signals without sync.
- Partial synchronization of buses (some bits synced, some not).
- Using output of sync chain in combinational logic.
- Skipping synchronizers in low-frequency test paths.
- Not constraining CDC paths as false in STA.

Q57: What is the role of CDC lint rules in RTL linting tools?

Answer:

Lint rules help:

- Enforce use of 2-flop sync for single-bit async inputs.
- Flag improper bus transfers across domains.
- Identify missing synchronizers.
- Warn against comb logic across clock domains.
- Enforce naming conventions for CDC signals.

This provides a first-level defense even before running CDC tools.



**Excellence in World class
VLSI Training & Placements**

Do follow for updates & enquires



+91- 9182280927